
La shell

La **shell** è un programma che permette all'utente di interagire con il sistema operativo attraverso **comandi testuali**. In pratica è il “traduttore” tra l'utente e il kernel.

La shell più diffusa nei sistemi UNIX-like è **BASH** (Bourne Again Shell), evoluzione della storica Bourne Shell.

Le funzioni principali che svolge una shell sono:

- mostrare il **prompt** e ricevere i comandi dall'utente;
- **interpretare ed eseguire** i comandi digitati (sia interni che esterni);
- effettuare sostituzioni automatiche (variabili d'ambiente, wildcard *, ecc.);
- gestire la **redirezione** dell'input e dell'output (es. > e <);
- eseguire **script**, cioè file contenenti sequenze di comandi;
- permettere il **controllo dei processi** (avvio in background, interruzione, sospensione...).

Una caratteristica molto importante della shell è che può essere **personalizzata**.

Ogni utente ha dei file di configurazione nella sua home (tipicamente .bashrc e .bash_profile) che vengono eseguiti automaticamente al login.

Da lì si possono impostare:

- prompt personalizzato,
- colori,
- alias (alias ll='ls -l'),
- variabili d'ambiente, ecc.

Inoltre, per ogni utente esiste anche un file chiamato **.bash_history**, che memorizza lo storico dei comandi eseguiti; il comando history lo legge proprio da questo file.

Nota: tutti questi file iniziano con . perché nei sistemi UNIX-like i nomi che iniziano con punto vengono considerati **nascosti**.

Il terminale

Il **terminale** è l'interfaccia (grafica o testuale) attraverso la quale l'utente interagisce con la shell.

All'inizio i terminali erano delle **telescriventi** (TTY), cioè macchine dotate di tastiera e stampante.

Quando sono arrivati i video, il concetto è rimasto lo stesso ma lo schermo ha sostituito la stampante.

Per questo ancora oggi nei sistemi Linux i terminali si chiamano **tty** e vengono numerati (tty1, tty2, ...).

Le distribuzioni Linux desktop avviano in genere **sette terminali virtuali**:

- i primi sei sono terminali **testuali**, con shell;
- il settimo ospita la **sessione grafica**.
Se apri una seconda sessione grafica, si userà tty8, e così via.

Per spostarsi fra i terminali si usano scorciatoie da tastiera:

- Alt + Fn → cambia terminale da sessione testuale;

- Ctrl + Alt + Fn → cambia terminale da sessione grafica.

Terminal Emulator

Nei sistemi grafici, non si va in console fisica, ma si apre un **terminal emulator**, cioè un programma che emula il comportamento di un terminale *dentro* una finestra della GUI.

Questi emulatori offrono in più anche funzioni grafiche come uso del mouse, selezione del testo, menu, ecc.

Esempi molto comuni:

Emulatore	Ambiente
xterm	ambiente grafico X (base)
Gnome Terminal	desktop GNOME
Konsole	desktop KDE

Il prompt dei comandi

Quando apri un terminale in Linux, la prima cosa che vedi è il **prompt**, cioè la stringa che la shell mostra in attesa che tu digiti un comando.

Nel caso di BASH, il prompt ha (di default) questa struttura:

name@host:workdir\$

- **name** → è il nome dell'utente connesso;
- **host** → è il nome del computer;
- **workdir** → indica in quale directory ti trovi (directory di lavoro);
- **\$** → indica che si tratta di un utente "normale". Se al posto del \$ compare #, significa che sei connesso come **root**, quindi con privilegi amministrativi.

Attenzione: questo è il **prompt di default**, ma può cambiare da distribuzione a distribuzione oppure essere **personalizzato** dall'utente tramite i file .bashrc e simili (ad esempio per mostrare ora, colori, nome della shell, ecc.).

Tipi di comandi eseguiti dalla shell

Quando scrivi qualcosa al prompt, la shell deve capire **che tipo di comando è**, e in base a questo decide come gestirlo:

Tipo di comando	Descrizione	Comportamento
Interno (built-in)	fa parte direttamente della shell	viene eseguito dalla shell stessa , senza creare nuovi processi

Tipo di comando	Descrizione	Comportamento
Eseguibile esterno	file binario (programma) presente sul disco	la shell avvia un nuovo processo figlio che esegue il programma
Script	file di testo contenente comandi shell	la shell lancia una nuova istanza della shell che interpreta lo script

👉 In pratica:

- prima la shell verifica se il comando è interno;
- se non lo è, **crea un processo figlio** e delega a lui l'esecuzione.

Esempio concreto:

- `cd` → è un comando **interno**
- `ls` → è un **programma esterno** (un binario in `/bin/ls`)
- `myscript.sh` → è uno **script**, viene interpretato da una nuova shell

Flag (opzioni dei comandi)

Le **flag** sono opzioni aggiuntive che modificano il comportamento di un comando. Sono molto comuni nella shell e sono **case-sensitive**: `-R` e `-r` non sono la stessa cosa.

Ogni comando ha le **proprie** flag (non esistono flag “universali”) e queste possono avere forme diverse:

Tipo	Esempio	Descrizione
Semplice	<code>-R</code>	una singola lettera preceduta da un trattino
Esteso	<code>--all</code> , <code>--no-spinner</code>	una parola (o più parole unite da <code>-</code>), preceduta da due trattini

Con argomento `-f file`, `--max-depth 3` la flag è seguita da un valore

Ci sono anche **flag letterali** (senza trattino) che si usano così come sono: dipende dal comando, quindi bisogna sempre verificare nelle pagine di manuale (`man`).

Quando vuoi usare **più flag semplici** puoi:

- scriverle separatamente: `-x -y -z`
- scriverle tutte insieme con un solo trattino: `-xyz`

Flag ricorrenti (che vedi spesso)

Flag	Significato
--help / -h	mostra l'help del comando
--verbose / -v	output più dettagliato (ripetibile: -vv, -vvv, ...)
--recursive / -r	esecuzione ricorsiva nelle sottodirectory
--force / -f	non chiedere conferma
--all / -a	mostra <i>tutto</i> (o il massimo possibile)
--version	mostra la versione del programma (solo comandi esterni)

Scorciatoie da tastiera utilizzate nella shell

La shell non si usa solo scrivendo comandi, ma anche sfruttando una serie di **scorciatoie** che velocizzano molto il lavoro.

Quelle più utili (e che devi assolutamente conoscere) sono queste:

Scorciatoia	Funzione
Tab	Completa automaticamente ciò che stai digitando. Se ci sono più possibilità, un secondo <i>Tab</i> mostra tutte le alternative.
Freccia su / giù	Scorri nell'elenco dei comandi già digitati in precedenza (history).
Ctrl + R	Ricerca interattiva nella history: scrivi una parte del comando e vengono mostrati i match.
Ctrl + H	Cancella il carattere a sinistra del cursore (equivalente a Backspace).
Ctrl + W	Cancella la parola a sinistra del cursore.
Ctrl + U	Cancella tutta la riga corrente.
Ctrl + L	Pulisce lo schermo (come un "clear").

 Nota: queste scorciatoie funzionano nella **Bash** e nella grande maggioranza delle altre shell posix-like.

Pagine di Manuale (man)

Scopo:

- Formattare e mostrare le pagine di guida online.
- Coprono comandi, funzioni di libreria, file speciali, ecc.

Database:

- Le informazioni sono memorizzate in un database generato con mandb.
- Comandi correlati:
 - whatis → descrizione breve di un comando.
 - apropos → ricerca per parola chiave.

Struttura tipica di una pagina man:

1. **NAME** → Nome comando + breve descrizione.
2. **SYNOPSIS** → Sintassi del comando.
3. **DESCRIPTION** → Descrizione dettagliata.
4. **OPTIONS** → Lista dei flag e spiegazioni.

Sintassi base:

man -[fkKw] [-S sezione] [sezione] [-P paginatore] <comando>

Flag principali di man

Flag	Funzione
-K	Cerca una stringa in tutte le pagine di manuale.
-w / --path	Mostra il percorso dei file contenenti la pagina man.
-P <paginatore>	Specifica il paginatore (default: less).
-Sx o x	Mostra le pagine della sezione x:
1 → comandi utente	
2 → system call (open, write, fork...)	
3 → funzioni libreria C (printf, scanf...)	
4 → file e dispositivi speciali (/dev/null, /dev/stdout...)	
-k	Equivalente a apropos.
-f	Equivalente a whatis.

Esempi pratici

- man printf → pagina di manuale del comando printf.
- man 3 printf o man -S3 printf → pagina della funzione printf della libreria C.

Nota:

- Se ci sono omonimi e non si specifica la sezione, viene mostrata la prima corrispondenza in ordine crescente di sezione.

Comando info

- **Scopo:** Mostra la documentazione dei comandi in formato **Info** (più dettagliata rispetto a man).
- **Sintassi:**

info <comando>

- **Esempio:**

info ls

Mostra la documentazione completa del comando ls in formato Info.

Comando whatis

- **Scopo:** Ricerca corrispondenze **esatte** della parola specificata in un database contenente brevi descrizioni dei comandi (NAME delle pagine man).
- **Sintassi:**

whatis <comando>

- **Esempio:**

whatis man

Output tipico:

man (1) - an interface to the on-line reference manuals

man (7) - macros to format man pages

Comando apropos

- **Scopo:** Simile a whatis, ma cerca **anche corrispondenze parziali** all'interno dei nomi e delle descrizioni dei comandi.
- **Sintassi:**

apropos <comando>

- **Esempio:**

apropos grep

Output tipico:

bzegrep (1) - search possibly bzip2 compressed files...

bzfgrep (1) - search possibly bzip2 compressed files...

bzgrep (1) - search possibly bzip2 compressed files...

egrep (1) - print lines matching a pattern
fgrep (1) - print lines matching a pattern
git-grep (1) - Print lines matching a pattern
grep (1) - print lines matching a pattern
...

Sintesi rapida:

Comando	Cosa fa	Ricerca	Output
info	Mostra documentazione dettagliata in formato Info –		Testo completo
whatis	Cerca corrispondenza esatta	Nome comando	Breve descrizione
apropos	Cerca corrispondenze parziali	Nome o keyword	Breve descrizione multipla

Comando history

- **Scopo:** Mostra lo **storico dei comandi** digitati dall'utente attualmente loggato.
 - **Dipende dalle variabili d'ambiente:**
 - HISTSIZE → massimo numero di comandi visualizzati (o cancellabili con -c).
 - HISTFILESIZE → numero di comandi salvati in ~/.bash_history.
-

Sintassi ed esempi

- Visualizzare la cronologia:

history

- Cancellare la cronologia:

history -c

Scorciatoie utili

Scorciatoia Funzione

!n	Esegue il comando numero n nella history.
!-n	Esegue il comando usato n comandi prima .
!!	Esegue l'ultimo comando (equivalente a !-1).
!stringa	Esegue il comando più recente contenente stringa .

Alias e gestione comandi

alias

- **Scopo:** Definisce un **alias**, un nome alternativo per un comando.
- **Priorità:** Gli alias hanno priorità sui comandi interni ed esterni.
- **Sintassi:**

alias nome=comando

- **Esempio:**

alias ls="ls -a"

unalias

- **Scopo:** Rimuove un alias.
- **Sintassi:**

unalias [nome | -a]

- **Flag:**
 - -a → elimina tutti gli alias.

type

- **Scopo:** Mostra informazioni sul tipo di comando.
- **Sintassi:**

type <comando> [comando] ...

- **Output tipico:** Indica se il comando è un **alias**, un **builtin** o un **comando esterno**.

Variabili e ambiente

Variabili Bash

- **Definizione:** Nome + valore.
- **Sintassi:**

A=34 # assegna valore

echo \$A # dereferenzia la variabile

- **Categorie:**
 1. **Variabili della shell** → disponibili solo nella shell corrente.
 2. **Variabili d'ambiente** → disponibili anche per comandi lanciati dalla shell.
- **Trasformare variabile in variabile d'ambiente:**

export VAR

Ambiente di un processo UNIX

- Insieme delle **variabili d'ambiente** visibili al processo.
-

Variabili comuni

Tipo	Variabile	Descrizione
Ambiente	PATH	Percorsi di ricerca dei comandi esterni
Ambiente	HOME	Directory home dell'utente
Ambiente	USER	Nome utente loggato
Ambiente	SHELL	Percorso della shell di default
Ambiente	PWD	Directory di lavoro corrente
Ambiente	OLDPWD	Directory precedente
Shell	UID	ID numerico dell'utente
Shell	GROUPS	ID dei gruppi dell'utente
Shell	PPID	PID del processo genitore della shell

Gestione variabili

set

- Mostra tutte le variabili della shell e d'ambiente e le funzioni definite.

unset

- Rimuove una o più variabili.
- **Sintassi:**

unset <var1> [var2] ...

export

- Esporta una variabile come variabile d'ambiente.
- **Sintassi:**

export VARIABILE[=valore]

printenv

- Mostra tutte le variabili d'ambiente o solo quelle specificate.
- **Sintassi:**

printenv [var1] [var2] ...

File system di linux ppt.2

Filesystem di Linux – Directory speciali . e ..

Concetti principali

- **.** (punto singolo):
 - Rappresenta la **directory corrente**.
 - Utile per riferirsi a file o eseguibili **nella stessa directory**.
- **..** (doppio punto):
 - Rappresenta la **directory padre** (quella che contiene la directory corrente).
 - Utile per salire di un livello nell'albero delle directory.
- **Nota:** La directory padre della radice / è se stessa (/.. ≡ /).

Esempi pratici

1. Usare . per riferirsi a file nella stessa directory

Supponiamo di avere un file script.sh nella directory corrente

./script.sh # esegue script.sh nella directory corrente

Senza ./, Bash cercherebbe il comando negli **eseguibili presenti in PATH**, quindi spesso è necessario usare ./ per lanciare file locali.

2. Usare .. per navigare verso la directory padre

cd .. # spostati nella directory padre

ls .. # lista i file della directory padre

3. Percorsi relativi con . e ..

- **Percorso relativo:** rispetto alla directory corrente.

Supponiamo struttura:

/home/vito/documenti/progetti/

cd /home/vito/documenti/progetti

Salire di un livello

cd .. # ora siamo in /home/vito/documenti

Tornare alla directory iniziale

cd ./progetti # equivalente a cd progetti

- **Combinazioni:**

cd ../../ # due livelli sopra la directory corrente

ls ../images # lista i file della directory "images" che sta un livello sopra

Riepilogo rapido

Simbolo	Significato	Esempio
.	Directory corrente	./file.txt
..	Directory padre	cd ..
/	Directory radice	/home/vito

Permessi dei file in Linux

Quando un file viene creato:

- Viene **associato** all'**utente** che lo ha creato.
- Viene associato al **gruppo principale** di quell'utente.

Su ogni file (e directory) possono essere attribuiti **tre tipi di permessi**:

Permesso Significato

Lettura possibilità di leggere il contenuto (read) **r**

Scrittura possibilità di modificare il contenuto (write) **w**

Esecuzione possibilità di eseguire il file (se è un programma o script) **x**

Questi permessi sono definiti **separatamente** per tre categorie di utenti:

Categoria Descrizione

i Tipo di file

u proprietario (user)

g gruppo del file (group)

o tutti gli altri utenti (other)

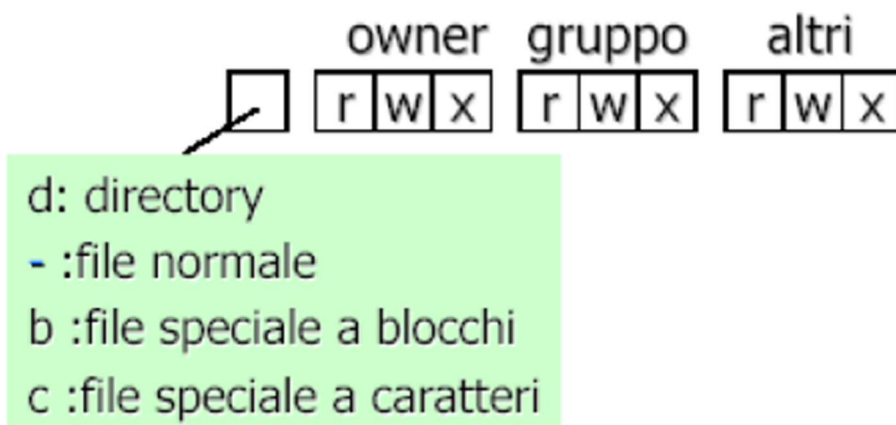
ACL – Access Control List

Ogni file/directory ha associata una **stringa di 10 caratteri**, visibile con `ls -l`, che rappresenta i permessi.

Esempio:

`-rwxr-xr--`

Posizione	Significato	Dettaglio
1° carattere	Tipo di file	- file normale / d directory / b blocchi / c caratteri / l link
2-4	Permessi del proprietario (u)	es. rwx
5-7	Permessi del gruppo (g)	es. r-x
8-10	Permessi degli altri (o)	es. r—



Tipi di file (1° carattere della ACL)

Carattere	Tipo di file	Descrizione
-	File normale	file di testo, script, binari, ecc.
d	Directory	cartella
b	Block device	dispositivi con I/O bufferizzato (es. HDD, DVD)
c	Character device	dispositivi con I/O non bufferizzato (es. terminali, audio, seriali)
l	Link simbolico	collegamento ad un file presente altrove

Significato dei permessi

Tipo di oggetto	r (read)	w (write)	x (execute)
File ordinari	leggere il contenuto	modificare il contenuto	eseguire (se programma/script)
Directory	vedere il contenuto (lista dei file)	aggiungere/rimuovere/rinominare file	<i>accedere</i> alla directory (entrarci e lavorare sui file al suo interno)
File speciali	leggere dal dispositivo (input)	scrivere sul dispositivo (output)	non ha significato

👉 **Nota:** l'assenza di un permesso viene indicata da -

Esempi di ACL

✅ **-rwx rw- r--**

Parte Significato

- File ordinario
- rwx proprietario → può leggere, scrivere ed **eseguire**
- rw- gruppo → può leggere e scrivere (ma **non** eseguire)
- r-- altri utenti → possono **solo** leggere

✅ **drwx r-x r-x**

Parte Significato

- d Directory
- rwx proprietario → può leggere, modificare e **accedere** alla directory
- r-x gruppo → può **leggere e accedere**, ma **non modificare**
- r-x altri utenti → stesso discorso

Permessi di default (quando si crea un file o directory)

Oggetto Permessi di default Significato

- File ordinari** -rw- r-- r-- proprietario può leggere/scrivere, altri solo leggere
- Directory** drwx r-x r-x proprietario può tutto, altri possono leggere ed accedere

chmod – Modifica dei permessi

- **Nome completo:** *change mode*
- Consente di **modificare i permessi** di accesso di file e directory.
- Può essere eseguito **solo dal proprietario del file** o da **root**.

Sintassi

chmod [-R] <mode> <file> [file] ...

Flag Significato

-R esegue l'operazione **ricorsivamente** su tutte le sottodirectory e i file contenuti

Mode – Forma simbolica

Formato generale:

<utenti><operatore><permessi>

Elemento	Valori possibili	Significato
----------	------------------	-------------

Utenti	u	proprietario
	g	gruppo
	o	altri
	a	tutti (equivalente a ugo)
Operatore	+	aggiungi permesso
	-	rimuovi permesso
	=	imposta esattamente quei permessi
Permessi	r	read
	w	write
	x	execute

Esempi

Comando	Descrizione
chmod g+w file.txt	Aggiunge il permesso di scrittura al gruppo
chmod ugo+rw file.txt (= a+rw)	Aggiunge lettura e scrittura a tutti
chmod o-x file.txt	Rimuove il permesso di esecuzione agli "altri"

Nota su ls -l (long format)

Il comando `ls -l` visualizza i file con **permessi, proprietario, gruppo, dimensione e data** in forma estesa.

Esempio

```
$ ls -l file.txt
```

```
-rw-r--r-- 1 vito vito 2345 Jul 10 15:42 file.txt
```

Significato:

Sezione	Spiegazione
---------	-------------

-rw-r--r--	ACL → tipo + permessi (file ordinario, u=rw, g=r, o=r)
------------	--

1	numero di hard link
---	---------------------

vito	proprietario
------	--------------

vito	gruppo
------	--------

2345	dimensione in byte
------	--------------------

Jul 10 15:42	data di modifica
--------------	------------------

file.txt	nome del file
----------	---------------

In pratica, `ls -l` è il modo principale per **verificare i permessi prima o dopo un `chmod`**.

chown – Change Owner

- Modifica **utente** e/o **gruppo** proprietario di uno o più file.
- Può essere usato **solo da root** (o con `sudo`).

Sintassi:

```
chown [-R] [utente][.][gruppo] <file> [file] ...
```

Flag Significato

-R applica la modifica **ricorsivamente** a tutte le sottodirectory/file

Esempi:

Comando	Descrizione
---------	-------------

<code>sudo chown utente1 file.txt</code>	cambia l' utente proprietario
--	--------------------------------------

<code>sudo chown -R utente1.gruppo1 directory</code>	cambia utente e gruppo per directory e contenuto
--	--

<code>sudo chown .gruppo1 file.txt</code>	cambia solo il gruppo proprietario
---	---

chgrp – Change Group

- Modifica **solo il gruppo** proprietario.
- Anche questo disponibile **solo per root**.

Sintassi:

chgrp [-R] <gruppo> <file> [file] ...

Esempio

Descrizione

sudo chgrp gruppo1 file.txt imposta gruppo proprietario su **gruppo1**

pwd – Print Working Directory

- Mostra il **percorso assoluto** della directory corrente.

Sintassi:

pwd

Esempio:

/home/user

cd – Change Directory

Cambia la **directory di lavoro corrente**.

Sintassi:

cd [percorso]

Comando Spiegazione

cd	va nella home dell'utente (equivalente a cd ~. USARE ~ QUANDO SI RIFERISCE A UTENTE ATTUALMENTE LOGGATO NEGLI ESERCIZI ALL'ESAME)
cd -	torna alla directory precedente
cd .	rimane nella directory corrente (nessun effetto)
cd ..	passa alla directory padre
Cd /	Vai nella radice del filesystem

★ ls - List

Descrizione:

Mostra i file e le directory presenti in un percorso. Permette di visualizzare anche file nascosti, ordinare per data, invertire l'ordine e ottenere informazioni dettagliate sui file.

Sintassi:

ls [flag] [percorso]

Tabella flag:

Flag	Descrizione
-a	Mostra tutti i file, inclusi quelli nascosti
-F	Accoda un identificativo di tipo a ciascun file (<i>* eseguibile, / directory, @ link simbolico</i>)
-r	Inverte l'ordine di visualizzazione delle righe
-R	Visualizza il contenuto delle subdirectory ricorsivamente
-t	Ordina i file per data di ultima modifica (dal più recente)
-u	Ordina i file per data di ultimo accesso (dal più recente)
-i	Mostra l'inode number del file
-1	Visualizza l'elenco in una singola colonna
-l	Mostra informazioni dettagliate (ACL, hard link, utente, gruppo, dimensione, ultima modifica, nome)

Tabella esempi:

Comando	Descrizione
ls -a	Mostra tutti i file, inclusi quelli nascosti
ls -l	Mostra informazioni dettagliate sui file
ls -lt	Ordina file per data di modifica con dettagli
ls -R	Visualizza ricorsivamente il contenuto delle subdirectory
ls -F	Aggiunge identificatori di tipo ai file

★ touch - Aggiorna il timestamp di un file

Descrizione:

Aggiorna la data e ora di accesso e modifica di un file. Se il file non esiste, lo crea vuoto.

Sintassi:

touch [flag] <percorso> ...

Tabella flag:

Flag Descrizione

- a Aggiorna solo il timestamp di accesso
- m Aggiorna solo il timestamp di modifica
- c Non crea alcun file se non esiste

Tabella esempi:

Comando Descrizione

- touch myfile Se esiste aggiorna timestamp, altrimenti crea il file
- touch -a myfile Aggiorna solo il timestamp di accesso
- touch -c myfile Aggiorna solo se esiste, non crea file
- touch -ca myfile Aggiorna solo l'accesso, non crea file

mkdir - Crea una nuova directory **make directory**

Descrizione:

Permette di creare directory singole o multiple, incluse eventuali directory padre mancanti.

Sintassi:

mkdir [flag] <percorso> ...

Tabella flag:

Flag Descrizione

- m Specifica i permessi della directory (Access Control List)
- p Crea directory padre mancanti senza errori
- v Modalità verbosa, stampa messaggi anche in caso di successo

Tabella esempi:

Comando Descrizione

- mkdir mydir Crea la directory mydir
- mkdir mydir1 mydir2 Crea più directory contemporaneamente
- mkdir -m 766 mydir Crea mydir con permessi 766
- mkdir -p mydir1/mydir2 Crea directory nidificate, creando mydir1 se non esiste

rmdir - Cancella una directory vuota remove directory

Descrizione:

Rimuove una directory solo se è vuota.

Sintassi:

rmdir [flag] <percorso> ...

Tabella flag:

Flag	Descrizione
--ignore-fail-on-non-empty	Sopprime messaggi di errore se la directory non è vuota
-v	Modalità verbosa, stampa messaggi anche in caso di successo

Tabella esempi:

Comando	Descrizione
rmdir mydir	Rimuove mydir se è vuota
rmdir -v mydir	Rimuove mydir e mostra conferma
rmdir --ignore-fail-on-non-empty mydir	Ignora errori se mydir non è vuota

cp - Copia file e directory

Descrizione:

Copia file e directory. Se la destinazione è una directory, il file mantiene il nome originale; altrimenti viene copiato con un nuovo nome.

Sintassi:

cp [flag] <percorso_origine> ... <percorso_destinazione>

Tabella flag:

Flag	Descrizione
-n	Non sovrascrivere file esistenti
-i	Chiede conferma prima di sovrascrivere
-f	Forza la copia, rimuovendo file non scrivibili
-r	Copia directory ricorsivamente (di base non copia directory)
-v	Modalità verbosa, mostra i file copiati

Tabella esempi:

Comando	Descrizione
cp file1 file2	Copia file1 in file2
cp -r dir1 dir2	Copia ricorsivamente dir1 in dir2
cp -i file1 file2	Chiede conferma se file2 esiste
cp -n file1 file2	Non sovrascrive file esistenti
cp -v file1 file2	Mostra il processo di copia

🌟 mv - Move files or directories

Descrizione:

Sposta file o directory. Rinominare un file è un caso speciale di spostamento. Di default sovrascrive i file esistenti.

Sintassi:

mv [flag] <percorso_origine> ... <percorso_destinazione>

Tabella flag:

Flag Descrizione

- i Chiede conferma prima di sovrascrivere file esistenti
- f Forza la sovrascrittura anche se non si possiedono permessi di scrittura
- v Modalità verbosa, mostra quali file vengono spostati

Tabella esempi:

Comando	Descrizione
mv file1 file2	Sposta o rinomina file1 in file2
mv -i file1 file2	Chiede conferma se file2 esiste
mv -v file1 dir/	Sposta file1 nella directory dir/ mostrando l'operazione

🌟 rm - Remove files or directories

Descrizione:

Rimuove file o directory. Di default, le directory vengono rimosse solo se vuote. Per cancellare directory con contenuto, usare il flag -r.

Sintassi:

rm [flag] <percorso> ...

Tabella flag:

Flag Descrizione

- r Cancella directory ricorsivamente, anche se non vuote
- i Chiede conferma per ciascun file rimosso
- f Ignora file non esistenti e non chiedere conferma
- v Modalità verbosa, mostra quali file vengono rimossi

Tabella esempi:

Comando Descrizione

- rm file1 Rimuove file1
- rm -r dir1 Cancella dir1 e tutto il suo contenuto
- rm -i file1 Chiede conferma prima di rimuovere file1
- rm -f file1 Rimuove file1 senza chiedere conferma
- rm -v file1 Mostra il file rimosso

file - Determine file type

Descrizione:

Determina il tipo di un file, ad esempio testo, binario, immagine, ecc.

Sintassi:

file [flag] <percorso> ...

Tabella flag:

(I flag principali dipendono dalla versione di file; spesso non sono obbligatori.)

Flag Descrizione

- b Mostra solo il tipo del file senza il nome
- i Mostra il MIME type del file

Tabella esempi:

Comando Descrizione

- file myfile Determina il tipo di myfile
 - file -b myfile Mostra solo il tipo di myfile senza nome
 - file -i myfile Mostra il MIME type di myfile
-

stat - Show file information

Descrizione:

Mostra informazioni dettagliate su un file, inclusi permessi, proprietario, dimensione e timestamp di creazione/modifica/accesso.

Sintassi:

stat [flag] <percorso> ...

Tabella flag:

Flag Descrizione

- c Permette di personalizzare il formato delle informazioni
- t Output compatto in una singola riga

Tabella esempi:

Comando	Descrizione
stat myfile	Mostra informazioni complete su myfile
stat -t myfile	Mostra informazioni compatte su una riga
stat -c "%s %y" myfile	Mostra solo dimensione e data di ultima modifica

Inode e struttura dei file system UNIX-like

- In UNIX ogni elemento del file system (file, directory, link, ecc.) è rappresentato da una **struttura dati chiamata inode**.
- Contenuto di un inode:**
 - Attributi: tempi di accesso e modifica, proprietario, permessi, link count, ecc.
 - Puntatori ai **data block**, dove è memorizzato il contenuto del file.
- Ogni inode ha un **numero identificativo unico (inode number)**, e c'è una corrispondenza biunivoca tra file e inode.
- Nei **directory block** (data block delle directory) sono presenti le **directory entry**, che associano a ciascun file o directory il relativo inode number e il nome.

Link ai file

Un riferimento a un file è detto **link**, e UNIX ne prevede due tipi: **hard link** e **soft link (symlink)**.

Hard link

- È un puntatore diretto all'inode di un file esistente.
- Creando un hard link si aggiunge una nuova **directory entry** per lo stesso inode.

- Caratteristiche principali:
 - Indistinguibile dal file originale, poiché è essenzialmente un altro nome per lo stesso inode.
 - Incrementa il **link count** dell'inode di 1.
 - Se un hard link viene rimosso, il link count diminuisce di 1.
 - Quando il link count raggiunge 0, l'inode viene deallocato e i data block liberati (file cancellato).
- Limitazioni:
 - Non si possono creare hard link su filesystem diversi.
 - La maggior parte dei sistemi UNIX non permette hard link a directory per evitare loop ricorsivi.

Soft link (symbolic link, symlink)

- È un file speciale che contiene il percorso assoluto di un altro file.
- Caratteristiche principali:
 - Ha un inode proprio, distinto da quello del file originale.
 - Creare o rimuovere un soft link non influisce sull'inode del file originale.
 - Non ha le limitazioni degli hard link (può puntare a file su altri filesystem e a directory).

Link e directory

- Una directory ha almeno **2 hard link**: uno alla directory stessa (.) e uno nella directory padre.
- Ogni sottodirectory creata genera automaticamente un hard link implicito (..).

In sintesi: gli **inode** gestiscono tutte le informazioni essenziali sui file, mentre **hard link** e **soft link** offrono due modi diversi di riferirsi a un file, con implicazioni diverse su visibilità, eliminazione e compatibilità tra filesystem.

In – Link

Crea collegamenti tra file. Di default crea **hard link**, ma può essere usato anche per creare **soft link (symlink)**.

Descrizione:

- Permette di creare un nuovo riferimento a un file esistente.
- Un **hard link** punta direttamente all'inode del file originale.
- Un **soft link** è un file speciale che contiene il percorso del file originale.

Sintassi:

In [flags] <percorso_origine> ... <percorso_destinazione>

Tabella Flag:

Flag Descrizione

-s Crea un **soft link** invece di un hard link.

Esempi:

Comando	Descrizione
In myfile1 myfile2	Crea un hard link di nome myfile2 a myfile1.
In -s myfile1 myfile2	Crea un soft link di nome myfile2 a myfile1.
In myfile1 myfile2 mydir	Crea hard link a myfile1 e myfile2 all'interno di mydir.

ESERCIZI DI FINE CAPITOLO

1. Spostarsi nella directory ***" /home/user/dir1/dir11"*** supponendo di trovarsi in ***" /home/user/dir2/dir21"***, utilizzando i percorsi assoluto e relativo.

Cd /home /user/dir1/dir11

Cd ../../dir1/dir11

2. Spiegare il significato dell'ACL ***"lrwxr-xr-x"*** relativa ad un file di nome ***"myfile"***.
 1. Convertire la componente di permessi dell'ACL in formato ottale.
 2. Scrivere il comando atto a rimuovere i privilegi di esecuzione per gli utenti diversi dal proprietario e non facenti parte del gruppo del file.
 3. Specificare quali siano i nuovi permessi in formato ottale.
3. Verificare in quale directory si trovi l'utente.
Successivamente, rinominare il file ***"myfile1"*** presente nella directory corrente in ***"myfile2"***, spostandolo nella directory padre della directory corrente.

Mv myfile1 ../myfile2

4. Usando la notazione letterale, si modifichino i permessi del file ***"myfile"*** in modo che risultino essere equivalenti a ***564***.

Chmod u=rx,g=rw,o=x myfile

5. Il comando `ls -F` produce il seguente output:

`file1 file2 file3 file4/`

Indicare quale dei seguenti comandi è esatto, motivando la risposta: **1)** `mv file1 file2 file3` **2)** `mv file1 file2 file4`

La 2 perché su `file4/` lo / indica una directory e in `mv` l'ultima parte ossia `file4` deve essere una directory

6. Dato il file "**myfile**" dotato dei permessi di accesso

"-r-xr--rwx", indicare come cambiano tali permessi quando vengono lanciati i seguenti comandi (**Nota:** specificarlo per ciascun passaggio):

1. `chmod 755 myfile`
2. `chmod g-r+w myfile`
3. `chmod o-x myfile`

1. -rwxr-xr-x

2. -rwx-wxr-x

3 -rwx-wxr—

7. Eseguire le seguenti operazioni:

1. Spostarsi nella home directory e creare le directory "**uno**" e "**due**".
2. Copiare il file `"/etc/profile"` nella directory "**uno**".
3. Copiare il file `"/etc/profile"` nella directory "**due**", cambiandone il nome in `"profile_copia"`.
4. Spostare il file `"profile"` nella directory "**due**" ed il file `"profile-copia"` nella directory "**uno**", rinominandolo in `"profile_bak"`.
5. Cancellare i due file con un solo comando.
6. Cancellare le due directory con un solo comando.

`Cd`

`Mkdir uno due`

`Cp /etc/profile uno`

`Cp /etc/profile due/profile_copia`

`Mv due/profile uno/profile_bak`

`rm uno/profile_bak due/profile`

`rmdir uno due ( rm -r uno due)`

8. Creare il seguente albero di directory e file all'interno della directory corrente utilizzando al più due comandi.

```
dir1
├── dir2
│   ├── file2.txt
│   └── file3.txt
├── dir3
│   └── file4.txt
└── file1.txt
```

Mkdir -p dir1/dir2 dir1/dir3

Touch dir1/file1.txt dir1/dir2/file2.txt dir1/dir2/file3.txt dir1/dir3/file4.txt

TERZO CAPITOLO

✦ Stream & Redirection – introduzione

In UNIX ogni processo ha **3 stream standard** associati, identificati da un **file descriptor** numerico:

FD	Nome stream	Valore di default
0	stdin (Standard input)	Tastiera
1	stdout (Standard output)	Terminale (schermo)
2	stderr (Standard error)	Terminale (schermo)

È possibile “deviare” uno di questi stream su un **file** invece che sul terminale, usando la sintassi:

comando [file_descriptor] <operatore> file

Se non specifichi esplicitamente il file descriptor, la shell assume quello **predefinito** per l'operatore.

Operatori principali

Operatore	Descrizione	File descriptor implicito
<	Redirezione dell' input	0 (stdin)
>	Redirezione dell' output in modalità truncate (sovrascrive il file)	1 (stdout)
>>	Redirezione dell' output in modalità append (accoda al file)	1 (stdout)
>&	Concatena /unisce due file descriptor (es: stderr → stdout)	1 (stdout)

ES.

Comando	Descrizione
<code>cmd > myfile.txt</code>	Redirige stdout in <code>myfile.txt</code> (modalità <i>truncate</i> → sovrascrive il contenuto).
<code>cmd 2>> myfile.txt</code>	Redirige stderr in <code>myfile.txt</code> in append (accoda senza sovrascrivere).
<code>cmd > myfile.txt 2>&1</code>	Redirige stdout in <code>myfile.txt</code> , e unisce stderr a stdout , quindi entrambi vengono scritti in <code>myfile.txt</code> .
⇒ <i>Equivalente a:</i> <code>cmd &> myfile.txt</code>	
<code>cmd < myin.txt > myout.txt 2> myerr.txt</code>	Redirige: stdin da <code>myin.txt</code> , stdout in <code>myout.txt</code> , stderr in <code>myerr.txt</code> .
<code>cmd < myin.txt >> myout.txt 2>&1</code>	stdin da <code>myin.txt</code> , stdout in <code>myout.txt</code> (append), e stderr unito a stdout → entrambi finiscono in <code>myout.txt</code> (in append).

Esecuzione multipla e pipeline di comandi

In Bash puoi eseguire **più comandi sulla stessa riga**, usando operatori che controllano **l'ordine** e **le condizioni di esecuzione**.

Operatori di sequenza

Sintassi	Significato
<code>comando1 ; comando2</code>	Esegue comando1 , poi comando2 (indipendentemente dal risultato del primo).
<code>comando1 && comando2</code>	Esegue comando1 , e solo se ha successo (exit code = 0) esegue anche comando2 . → AND logico
<code>`comando1`</code>	

Pipeline

Sintassi	Descrizione
----------	-------------

``comando1 comando2``

Questo meccanismo si chiama **pipeline** e ti permette di concatenare più comandi per creare veri e propri **flussi di elaborazione**.

Esempio classico:

```
cat file.txt | grep "ciao" | wc -l
```

 Qui l'output di `cat` va in `grep`, e l'output di `grep` va in `wc -l`.

🌟 echo – Print message to standard output

Descrizione: stampa una stringa sullo **stdout**. Può essere usato anche insieme alla redirectione per scrivere il testo in un file.

Sintassi:

echo [stringa]

Esempi:

Comando	Descrizione
echo "Hello, world!"	Stampa il messaggio su stdout
echo ~	Stampa il percorso della home dell'utente
echo "Hello, world!" > myfile.txt	Scrive il messaggio in myfile.txt (sovrascrivendo il file)
echo "Hello, world!" >> myfile.txt	Accoda il messaggio in myfile.txt

🌟 cat – Concatenate files

Descrizione: concatena il contenuto di uno o più file e lo invia su **stdout**. Usato anche per unire file o visualizzarne il contenuto.

Sintassi:

cat [flags] [percorso] ...

Tabella Flag:

Flag Descrizione

-n Numera le righe del contenuto in output

Esempi:

Comando	Descrizione
cat myfile1.txt myfile2.txt	Concatena myfile1.txt e myfile2.txt e li stampa su stdout
cat myfile1.txt	Stampa il contenuto di myfile1.txt
cat myfile1.txt myfile2.txt > myfile3.txt	Concatena myfile1.txt e myfile2.txt e scrive il risultato in myfile3.txt

🌟 more / less – Paginazione interattiva

Descrizione:

more e less sono programmi che permettono di visualizzare il contenuto di un file in maniera **interattiva**, mostrandolo una pagina alla volta.

Comando	Descrizione
---------	-------------

- `more` è la versione più semplice e storica (scorrimento solo **in avanti**)
- `less` è più moderno e potente: permette di **scorrere anche indietro**, effettuare **ricerche**, muoversi rapidamente, ecc.
👉 In pratica, oggi si usa quasi sempre **less**.

Sintassi:

`more <percorso>`

`less <percorso>`

Esempi:

Comando	Descrizione
---------	-------------

`less myfile.txt` Visualizza `myfile.txt` con paginazione interattiva

``ls -la`` `less``

🔗 **sort – Ordina righe di file**

Descrizione:

Ordina le righe di uno o più file.

Di default usa l'**ordine lessicografico** (alfabetico).

Sintassi:

`sort [flags] <percorso> ...`

Tabella flag:

Flag	Descrizione
------	-------------

`-c` Verifica se il file è ordinato; se non lo è, mostra la prima riga fuori ordine

`-n` Ordina secondo il **valore numerico**

`-f` Ordina in modo **case-insensitive** (ignora maiuscole/minuscole)

`-r` **Inverte l'ordinamento**

`-k N` Ordina secondo il **campo N-esimo** (default separatore: spazio; modificabile con `-t`)

🔗 head – Show beginning of file

Descrizione:

Mostra le **prime righe** o i **primi byte** di un file.
Di default visualizza le **prime 10 righe**.

Sintassi:

head [flags] <percorso>

Tabella flag:

Flag	Descrizione
------	-------------

- | | |
|-------|---|
| -c N | Mostra i primi N byte del file |
| -c -N | Mostra i byte iniziali escludendo gli ultimi N |
| -n N | Mostra le prime N righe |
| -n -N | Mostra le prime righe escludendo le ultime N |

🔗 tail – Show end of file

Descrizione:

Mostra le **ultime righe** (o byte) di un file.
Per default visualizza **le ultime 10 righe**.

Sintassi:

tail [flags] <percorso>

Tabella Flag:

Flag	Descrizione
------	-------------

- | | |
|-------|---------------------------------------|
| -c N | Mostra gli ultimi N byte |
| -c +N | Mostra i byte a partire da N |
| -n N | Mostra le ultime N righe |
| -n +N | Mostra le righe a partire da N |

🔗 uniq – Remove or show duplicated lines

Descrizione:

Mostra o elimina le **righe duplicate** all'interno di un file.

⚠ *Funziona solo se le righe duplicate sono **adiacenti** → spesso si usa assieme a sort., prima sort e poi uniq*

Sintassi:

uniq [flags] <percorso>

Tabella Flag:**Flag Descrizione**

- d Mostra **solo** le righe duplicate
 - u Mostra **solo** le righe **uniche**
 - c Precede ogni riga con il **numero di occorrenze**
 - i Confronto **case-insensitive**
-

🔧 wc – Word count**Descrizione:**

Conta **righe, parole, caratteri e byte** contenuti in uno o più file.

Se non è specificato nessun flag, mostra **righe, parole e byte**.

Sintassi:

wc [flags] <percorso>

Tabella Flag:**Flag Descrizione**

- l Conta **le righe**
- w Conta **le parole**
- m Conta **i caratteri**
- c Conta **i byte**

Output formato: [righe] [parole] [caratteri] [byte] percorso

🔧 cut – Extract parts of each line**Descrizione:**

Estrae **byte, caratteri o campi** specifici da ogni riga di testo di un file.

Sintassi:

cut [flags] <percorso>

Tabella Flag:

Flag Descrizione

- b Estrae i **byte** indicati dal range
- c Estrae i **caratteri** indicati dal range
- f Estrae i **campi** indicati dal range
- d Specifica il **separatore di campi** (default: TAB)

Range possibili:

Forma Significato

- N solo l'elemento N
- N-M dal N-esimo al M-esimo
- N- dal N-esimo alla fine
- N dall'inizio fino all'N-esimo

diff – Compare files or directories

Descrizione:

Confronta due file o directory e mostra le **differenze**.
Molto utile per confrontare file di configurazione.
(Esiste anche colordiff per un output colorato).

Sintassi:

diff [flags] <percorso_1> <percorso_2>

Tabella Flag:

Flag	Descrizione
-r	Confronta directory ricorsivamente
--brief	Indica solo se ci sono differenze , senza mostrarle
--ignore-case	Confronto case-insensitive

Esempi – tail

Comando	Descrizione
s file.txt	Mostra le ultime 10 righe di file.txt
tail -n 5 file.txt	Mostra le ultime 5 righe del file

Comando	Descrizione
---------	-------------

tail -n +6 file.txt	Mostra le righe a partire dalla numero 6
---------------------	---

tail -c 20 file.txt	Mostra gli ultimi 20 byte
---------------------	----------------------------------

tail -c +30 file.txt	Mostra i byte a partire dal 30° byte
----------------------	---

✓ Esempi – uniq

Comando	Descrizione
---------	-------------

uniq file.txt	Mostra le righe del file eliminando i duplicati adiacenti
---------------	--

uniq -d file.txt	Mostra solo le righe che sono duplicate
------------------	---

uniq -u file.txt	Mostra solo le righe uniche (non ripetute)
------------------	--

uniq -c file.txt	Mostra le righe precedute dal numero di occorrenze
------------------	---

uniq -i file.txt	Confronta le righe ignorando maiuscole/minuscole
------------------	---

✓ Esempi – wc

Comando	Descrizione
---------	-------------

wc file.txt	Conta righe, parole e byte di file.txt
-------------	--

wc -l file.txt	Conta solo le righe
----------------	----------------------------

wc -w file.txt	Conta solo le parole
----------------	-----------------------------

wc -m file.txt	Conta i caratteri
----------------	--------------------------

wc -c file.txt	Conta i byte
----------------	---------------------

✓ Esempi –

Comando	Descrizione
---------	-------------

cut -b 1-3 file.txt	Estrae i byte da 1 a 3 di ogni riga
---------------------	--

cut -c 5-10 file.txt	Estrae i caratteri da 5 a 10 di ogni riga
----------------------	--

cut -f 2 dati.csv	Estrae il 2° campo da dati.csv (TAB-separato)
-------------------	--

cut -f 1,3 -d ';' dati.csv	Estrae il 1° e 3° campo , usando ; come separatore
----------------------------	---

✓ Esempi – diff

Comando	Descrizione
diff file1.txt file2.txt	Mostra le differenze tra file1.txt e file2.txt
diff -r cartella1 cartella2	Confronta ricorsivamente due directory
diff --brief file1.txt file2.txt	Indica solo se i file sono diversi , senza mostrare le differenze
diff --ignore-case file1.txt file2.txt	Confronta i file ignorando le differenze di maiuscole/minuscole

🌟 Esempi – head

Comando	Descrizione
head file.txt	Mostra le prime 10 righe di file.txt
head -n 5 file.txt	Mostra le prime 5 righe del file
head -n -3 file.txt	Mostra le prime righe, escludendo le ultime 3
head -c 20 file.txt	Mostra i primi 20 byte
head -c -10 file.txt	Mostra i byte iniziali escludendo gli ultimi 10

ESERCIZI

1. Dato il file *elenco.txt*, scrivere una riga di comando atta a visualizzare:

- La terza e la quarta riga.
- Le penultime tre righe.
- L'N-esima riga.

Head -4 elenco.txt | tail -2

Tail -4 | head -3

Head -N | tail -1

2. Scrivere la pipeline di comandi atta a visualizzare il nome dell'ultimo file nella directory di lavoro in ordine lessicografico.

Ls | sort | tail -1

3. Dato il file *elenco.txt*, scrivere una riga di comando atta a creare un file *nome.txt*, contenente la terza parola della seconda riga del file in ordine lessicografico.

```
Sort -k 3 elenco.txt | Head -2 | tail -1 | cut -f 2 -d " " > nome.txt
```

Scrivere una riga di comando atta a rimuovere la prima riga dal file *elenco.txt*, salvandone il contenuto.

```
Tail -n +2 elenco.txt > temp.txt && mv temp.txt elenco.txt
```

✦ Espressioni regolari (Regex)

◆ Descrizione:

Un'espressione regolare (regex) è un pattern che identifica un insieme di stringhe.

◆ Sintassi base:

Sintassi Significato

a una singola occorrenza del carattere a

[abcd] un carattere tra quelli elencati

[a-d] un carattere nel range (da a a d)

[^abcd] un carattere **non** tra quelli elencati

. qualsiasi carattere singolo

a* zero o più a

a+ una o più a

a? al massimo una a

a{N} esattamente N occorrenze

a{N,} almeno N occorrenze

a{,N} al massimo N occorrenze

a{N,M} tra N ed M occorrenze

◆ Posizionamento:

Sintassi Significato

`^a` a all'inizio riga

`a$` a a fine riga

`\<a` a all'inizio parola

`a\>` a a fine parola

``regex1 regex2``

◆ Gruppi:

Simbolo BRE (grep)		ERE (grep -E)	Esempio
<code>*</code>	0 o più ripetizioni	idem	<code>a* → "", a, aa, aaa</code>
<code>\+</code>	1 o più ripetizioni	<code>+</code>	<code>a\+ / a+ → a, aa, aaa</code>
<code>\?</code>	0 o 1 volta	<code>?</code>	<code>a\? / a? → "" o a</code>
<code>\{n\}</code>	esattamente n volte	<code>{n}</code>	<code>[0-9]\{3\} / [0-9]{3} → 3 cifre</code>
<code>\{n,\}</code>	almeno n volte	<code>{n,}</code>	<code>a\{2,\} → aa, aaa, aaaa</code>
<code>\{m,n\}</code>	tra m e n volte	<code>{m,n}</code>	<code>[0-9]\{2,4\} / [0-9]{2,4} → 2, 3 o 4 cifre</code>

◆ Classi di caratteri: (da usare in [...])

Classe Significato

`[:upper:]` lettere maiuscole

`[:lower:]` lettere minuscole

`[:digit:]` cifre numeriche

`[:alpha:]` caratteri alfabetici

`[:alnum:]` alfanumerici

`[:space:]` ogni spazio (incl. newline)

`[:blank:]` spazi e tabulazioni

`[:punct:]` punteggiatura

✦ grep – Global Regular Expression Print

Descrizione:

Mostra le righe che corrispondono ad una regex.

Sintassi:

grep [flags] pattern <file>

Flag principali:

Flag Significato

- E usa **regex estese** (+, ?, {}, `
- c mostra solo il **numero di righe** che batchano
- n aggiunge il **numero di riga** ad ogni match
- v mostra le righe che **non** corrispondono
- i match **case-insensitive**

Perfetto 🤓 allora ti faccio una **tabella di esempi reali di grep con file.txt** (quello con i numeri e i nomi da *one a fifteen*) e una breve descrizione per ogni flag.

 Contenuto di file.txt (riferimento):

15 fifteen
14 fourteen
13 thirteen
12 twelve
11 eleven
10 ten
9 nine
8 eight
7 seven
6 six
5 five
4 four
3 three
2 two
1 one

💡 Esempi grep

Comando	Output	Descrizione
grep 'ee' file.txt	15 fifteen 14 fourteen 13 thirteen 3 three	Trova tutte le righe con "ee" dentro.
grep -E 'e{2}' file.txt	15 fifteen 14 fourteen 13 thirteen 3 three	Stesso di sopra, ma usando regex estesa con -E.
grep -c 'e' file.txt	12	Conta quante righe contengono almeno una "e".
grep -n 'ten' file.txt	6:10 ten	Mostra la riga (6ª) dove compare "ten", con numero riga.
grep -v 'teen' file.txt	12 twelve 11 eleven 10 ten 9 nine 8 eight 7 seven 6 six 5 five 4 four 3 three 2 two 1 one	Mostra tutte le righe che NON contengono "teen".
grep -i 'FIVE' file.txt	5 five	Cerca senza distinzione maiuscole/minuscole .

Comando	Output	Descrizione
grep '[24]' file.txt	14 fourteen 12 twelve 4 four 2 two	Trova righe che contengono 2 o 4 .
grep '^1.*e\$' file.txt	12 twelve 1 one	Cerca righe che iniziano con 1 e finiscono con e .
grep '\<f[a-z]+n\>' file.txt	15 fifteen 14 fourteen	Matcha parole che iniziano con f , finiscono con n , e hanno lettere in mezzo.
grep -E '^1[04-9].*te+n\$' file.txt	15 fifteen 14 fourteen 10 ten	Cerca righe che iniziano con 10–15 e contengono te+n (es: "ten", "teen").
grep 'e\{2\}' file.txt	15 fifteen 14 fourteen 13 thirteen 3 three	Trova parole con due "e" consecutive .
grep '[:digit:]' file.txt	tutte le righe	Usa una classe di caratteri : matcha righe con numeri.
grep '^[:alpha:]' file.txt	(nessun output)	Cerca righe che iniziano con una lettera (qui tutte iniziano con numeri, quindi niente).

find

 Cerca file o directory ricorsivamente in un percorso.
Supporta pattern (*, ?, [abc]) e regex.

 **Sintassi:**

find <percorso> [flags]

📌 Flag principali:

- -name <pattern> → cerca file col nome che matcha il pattern.
 - -regex <regex> → cerca file col percorso che matcha la regex.
 - per regex estese: -regextype posix-extended.
 - -user <nome> / -group <nome> → filtra per proprietario utente/gruppo.
 - -type <tipo> → filtra per tipo (f file, d dir, l link simbolici).
 - /
 - -size [+|-]N[bckMG] → filtra per dimensione (N blocchi da 512B, o con B/KB/MB/GB).
 - +N → più grande di N
 - -N → più piccolo di N
 - N → esattamente N
-

📌 sed ^

📄 Stream editor → trasforma il contenuto di file o stream di testo.

📌 Sintassi:

sed [flags] <percorso>

📌 Flag principali:

- -e <script> → specifica lo script da eseguire.

📌 Esempi classici:

- sed -e '/regex/d' file.txt → elimina le righe che matchano la regex.
 - sed -e user file.txt → elimina le righe dalla N alla M.
 - sed -e '/regex/!d' file.txt → elimina tutte le righe **che non** matchano la regex.
 - sed -e 's/regex/sostituzione/' file.txt → sostituisce la prima occorrenza per riga.
 - sed -e 's/regex/sostituzione/g' file.txt → sostituisce tutte le occorrenze.
-

👤 Utenti e gruppi

- **Utenti:**
 - root → superutente con tutti i privilegi.
 - utenti regolari → con privilegi limitati.
- **Gruppi:**

- ogni utente ha **un gruppo primario** (usato nei file creati).
 - può appartenere a più **gruppi secondari**.
 - info in /etc/group, formato:
 - group:password:gid:user1,...,userN
 - group → nome gruppo
 - password → solitamente vuoto
 - gid → ID numerico gruppo
 - user1,...,userN → utenti che appartengono al gruppo
-

who, whoami, groups

🔪 **who** → mostra utenti loggati e info

- who -a → info estese
- who -l → processi di login
- who -q → solo nomi utenti + conteggio
- who -u → utenti loggati + idle time + PID shell

🔪 **whoami** → stampa l'utente corrente.

🔪 **groups [utente]** → mostra gruppi a cui appartiene l'utente (default = corrente).

id, last

🔪 **id [utente]** → mostra ID utente + gruppi

- -g → solo ID gruppo primario
- -G → solo ID gruppi secondari
- -u → solo ID utente
- -n → con i precedenti → mostra nomi invece che numeri

🔪 **last** → mostra ultimi login/logout + riavvii (legge /var/log/wtmp).

- Info: utente, terminale, host, data login, durata sessione.
-

su, sudo

🔪 **su [utente]** → cambia utente (default = root).

- richiede password dell'utente di destinazione.

🔪 **sudo [flags] comando** → esegue comando come altro utente (default root).

- -H → mantiene la home dell'utente corrente
 - -u user → esegue come utente specifico
-

date

🔴 **date [flags] [data]** → mostra o imposta data/ora.

- solo root può modificarla.

🔴 **Flag:**

- -d <data> → mostra data/ora corrispondenti a stringa data
- -r <file> → mostra ultima modifica del file
- -s <data> → imposta data/ora

🔴 **Esempio:**

```
date -s "2020-12-31 23:45"
```

→ imposta data/ora di sistema.

shutdown

- **Cosa fa:** spegne il sistema (solo root può eseguirlo).
 - **Avviso:** tutti gli utenti loggati ricevono un messaggio prima dello spegnimento.
-

Sintassi

```
shutdown [flags] <time> [message]
```

⚡ **Flag**

- -k → **non spegne**, manda solo messaggio di avviso agli utenti.
 - -r → riavvia il sistema dopo lo shutdown.
 - -h → spegne il sistema.
 - -t <N> → attende **N secondi** prima di spegnere.
-

Formati per

- hh:mm → spegne all'orario specificato (es. 23:00).
- +N → spegne dopo **N minuti** (es. +10).
- now → spegne subito.

Perfetto 👍 ti preparo anche questo blocco nello stesso **formato riassuntivo** che stiamo facendo, così hai tutto pulito e coerente.

passwd

- **Cosa fa:** cambia la password dell'utente corrente.
 - **Funzionamento:**
 - Prima chiede la **password attuale**.
 - Poi richiede la nuova e ne valuta la complessità.
-

File /etc/passwd

Contiene info sugli account utente.

Struttura:

user:password:uid:gid:info:home:shell

- **user** → nome utente.
 - **password** → hash cifrato o "x" se usato /etc/shadow.
 - **uid** → ID numerico dell'utente.
 - **gid** → ID numerico del gruppo principale.
 - **info (GECOS)** → info aggiuntive sull'utente.
 - **home** → directory home.
 - **shell** → shell di login.
-

File /etc/shadow

Contiene gli **hash delle password** (più sicuro di /etc/passwd).

Struttura:

user:password:last_change:min:max:warn:inactive:expire

- **user** → nome utente.
- **password** → hash (vuoto = nessuna password; *, !, !! = login bloccato).
- **last_change** → giorni dall'epoch UNIX (1/1/1970) all'ultima modifica.
- **min** → giorni minimi prima di poter cambiare di nuovo la password.
- **max** → durata massima della password (in giorni).
- **warn** → giorni di preavviso prima della scadenza.

- **inactive** → giorni dopo la scadenza prima di disabilitare l'account.
 - **expire** → data di scadenza dell'account.
-

useradd

- **Cosa fa:** crea un nuovo utente o modifica i **default** per i nuovi utenti.

Sintassi

useradd [flags] [user | -D]

Flag principali

- -D → aggiorna i **default**, non crea l'utente.
 - -d <home_dir> → specifica home directory.
 - -m → crea la home se non esiste.
 - -g <group> → gruppo principale (già esistente).
 - -G <group1,...> → gruppi supplementari.
 - -p <password> → hash della password (va generato con crypt).
 - -s <shell> → shell di login.
 - -u <uid> → specifica UID.
-

userdel

- **Cosa fa:** elimina un utente.
 - **Sintassi:**
 - userdel [flags] <user>
 - **Flag:**
 - -r → rimuove anche i file nella home directory.
-

groupadd

- **Cosa fa:** crea un nuovo gruppo.
 - **Sintassi:**
 - groupadd [flags] <group>
 - **Flag:**
 - -g <gid> → specifica GID.
-

groupdel

- **Cosa fa:** elimina un gruppo.
 - **Nota:** non è possibile rimuovere un gruppo se è **gruppo primario** di un utente esistente.
 - **Sintassi:**
 - groupdel <group>
-

Es. voglia l'hash della password di ivano nello shadow

```
ivano@debian-ivano:~/so$ grep '^ivano:' ../shadow_example
ivano:<password-hash>:18979:0:99999:7:::
ivano@debian-ivano:~/so$ grep '^ivano:' ../shadow_example | cut -f 2 -d ':'
```

Perfetto  andiamo avanti con la stessa formula schematica chiara e compatta che stiamo usando!

Archiviazione e Filesystem

Periferiche

- Rappresentate come **file speciali a blocchi** in /dev.
- Convenzione dei nomi:
 - **Prime 2 lettere** → tipo e canale.
 - **Terza** → numero del device sul canale.
 - **Quarta** → numero di partizione.

Esempi:

- /dev/hda → primo disco PATA master.
 - /dev/hdd3 → terza partizione del disco slave sul secondo canale PATA.
 - /dev/sdb1 → prima partizione del secondo disco SATA.
-

Filesystem globale

- Tutti i filesystem devono essere **montati** a partire da / (root).
 - La **partizione principale** viene montata all'avvio.
 - Molte distro montano unità esterne in /mnt o /media.
-

mtab e fstab

/etc/mtab

- Elenco dei filesystem **già montati**, con relative opzioni.

/etc/fstab

- Contiene i **mount point statici** montati al boot.
- noauto → impedisce montaggio automatico.

Struttura riga:

device mount_point fs_type options dump passno

- **device** → file in /dev relativo al device.
- **mount_point** → directory di mount.
- **fs_type** → tipo di filesystem.
- **options** → es. ro, rw, noauto.
- **dump** → 0=nessun backup, 1=backup abilitato.
- **passno** → ordine di controllo con fsck (0=salta).

mount / umount

mount

- Monta un filesystem.
- Se presente in /etc/fstab, basta il nome del device o mount point.
- Senza argomenti → mostra i device montati.

Sintassi:

mount [flags] [device] [directory]

Flag:

- -a → monta tutto da /etc/fstab.
- -o <options> → specifica opzioni (ro, rw, noauto...).
- -t <type> → forza tipo di filesystem (altrimenti autodetect).

umount

- Smonta un filesystem.
- **Sintassi:**

umount [device | mount_point]

df (disk free)

- Mostra lo **spazio utilizzato e libero** nei filesystem montati.

- Senza argomenti → info di tutti i filesystem montati.

Sintassi:

df [flags] [file] ...

Flag:

- -i → mostra utilizzo degli inode.
-

du (disk usage)

- Mostra lo **spazio occupato dai file o directory** (ricorsivo).
- Senza argomenti → spazio della directory corrente.

Sintassi:

du [flags] [file] ...

Flag:

- -a → include anche i singoli file, non solo directory.
 - -B N → misura blocchi di N byte (default 1024 = kB).
 - -d N → limita la profondità massima di analisi a N directory.
-

Gestione dei Job e Processi

◆ **Concetti base**

- **Job** → pipeline di comandi gestita dalla shell.
- **Foreground (default)** → shell attende il termine (sincrono).
- **Background (&)** → shell libera subito il prompt (asincrono).
- All'avvio di un job in background vengono mostrati:
 - **jobspec** (numero job).
 - **PID** dell'ultimo processo associato.

Scorciatoie

- **CTRL+Z** → sospende job in foreground (lo sposta in background).
 - **CTRL+C** → termina job in foreground.
-

sleep, yes

- **sleep <N>** → sospende la shell per N secondi.
- **yes [stringa]** → ripete all'infinito una stringa (default: y).

fg, bg, jobs

- **fg [jobspec]** → porta un job in foreground (default ultimo).
- **bg [jobspec]** → riprende un job sospeso in background.
- **jobs [flags] [jobspec]** → mostra lo stato dei job.
 - -l → info estese.
 - -p → solo PID.



- Mostra i processi attivi.

Sintassi: ps [flags]

- **UNIX style:**
 - -u [utente] → processi di utente.
 - -p [pid] → processo con PID specifico.
 - -f → output esteso.
- **BSD style:**
 - r → solo processi in esecuzione.
 - a → include processi di altri utenti (con terminale).
 - x → include processi senza terminale.
 - u → output user-oriented.

Struttura ps

PID TTY TIME CMD

top

- Vista **interattiva e in tempo reale** dei processi attivi.

kill / killall

- **kill [flags] <PID>** → invia segnali a un processo (default SIGTERM 15).
- **killall [flags] <nome>** → segnali a processi con quel nome.

Flag:

- -s <segnale> → specifica il segnale (numero o nome).

- **-u [utente]** (solo killall) → limita a processi di un utente.
 - **-i** (solo killall) → chiede conferma prima di agire.
-

Segnali principali

- **1 SIGHUP** → terminale disconnesso.
- **2 SIGINT** → interruzione (CTRL+C).
- **3 SIGQUIT** → terminazione con core dump.
- **6 SIGABRT** → terminazione per errore critico.
- **9 SIGKILL** → uccisione forzata (non ignorabile).
- **11 SIGSEGV** → errore di memoria (segmentation fault).
- **15 SIGTERM** → richiesta di terminazione (default).
- **19 SIGSTOP** → sospensione (CTRL+Z).